Program 1

8-Puzzle Problem Using Best First Search

Solve the 8-puzzle using Greedy Best First Search. Use the heuristic h(n) = number of misplaced tiles. Generate moves and print the sequence of states from the initial configuration to the goal.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q1.py > ...

```python
from queue import PriorityQueue

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

initial = (1, 2, 3,
           4, 0, 6,
           7, 5, 8)

moves = {
    "Up": -3,
    "Down": 3,
    "Left": -1,
    "Right": 1
}

def heuristic(state):
    count = 0
    for i in range(9):
        if state[i] != 0 and state[i] != goal[i]:
            count += 1
    return count

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)

    row = zero // 3
    col = zero % 3

    possible = []

    if row > 0:
        possible.append(("Up", zero - 3))
    if row < 2:
        possible.append(("Down", zero + 3))
    if col > 0:
        possible.append(("Left", zero - 1))
    if col < 2:
        possible.append(("Right", zero + 1))

    for move, pos in possible:
        new_state = list(state)
        new_state[zero], new_state[pos] = new_state[pos], new_state[zero]
        neighbors.append((move, tuple(new_state)))

    return neighbors

pq = PriorityQueue()
pq.put((heuristic(initial), initial, []))

visited = set()

while not pq.empty():
    h, current, path = pq.get()

    if current in visited:
        continue

    visited.add(current)

    if current == goal:
        print("Initial:", initial, "h =", heuristic(initial))
        for i, (move, state) in enumerate(path, start=1):
            print(f"Move {i}: {move} -> {state} h={heuristic(state)}")
        print("\nGOAL REACHED")
        break

    for move, neighbor in get_neighbors(current):
        if neighbor not in visited:
            new_path = path + [(move, neighbor)]
            pq.put((heuristic(neighbor), neighbor, new_path))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur
.py"
Initial: (1, 2, 3, 4, 0, 6, 7, 5, 8) h = 2
Move 1: Down -> (1, 2, 3, 4, 5, 6, 7, 0, 8) h=1
Move 2: Right -> (1, 2, 3, 4, 5, 6, 7, 8, 0) h=0

GOAL REACHED
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 2

A* Algorithm on a Weighted Graph

Implement A* using f(n) = g(n) + h(n) on the same weighted graph from Lab 6. Find the OPTIMAL path from S to G. Print the explored order, optimal path, and total cost.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q2.py > astar

```python
import heapq
graph = {
    'S': [('A', 3), ('B', 5)],
    'A': [('C', 4), ('D', 2)],
    'B': [('D', 6), ('G', 9)],
    'C': [('G', 7)],
    'D': [('E', 1)],
    'E': [('G', 3)],
    'G': []
}
heuristic = {
    'S': 10,
    'A': 7,
    'B': 6,
    'C': 5,
    'D': 4,
    'E': 2,
    'G': 0
}
def astar(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0, start, []))

    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)

        path = path + [node]

        print("Visited:", node)

        if node == goal:
            print("\nOptimal Path:", " -> ".join(path))
            print("Total Cost:", g)
            return

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    pq,
                    (new_f, new_g, neighbor, path)
                )

astar('S', 'G')
```

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/
Visited: S
Visited: A
Visited: D
Visited: E
Visited: G

Optimal Path: S -> A -> D -> E -> G
Total Cost: 9
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 3

AO* Algorithm on an AND-OR Graph

Implement AO* on the given AND-OR graph. Find the best solution sub-graph by repeatedly choosing the cheapest connector and revising heuristic estimates as nodes are expanded.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q3.py > ao_star

```python
graph = {
    'A': [
        ('OR', ['B'], 1),
        ('AND', ['C', 'D'], 1)
    ],

    'B': [
        ('AND', ['E', 'F'], 1)
    ],

    'C': [],
    'D': [],
    'E': [],
    'F': []
}
heuristic = {
    'A': 0,
    'B': 5,
    'C': 3,
    'D': 4,
    'E': 7,
    'F': 9
}
def ao_star(node):
    if graph[node] == []:
        return 0

    costs = []

    for connector_type, children, edge_cost in graph[node]:

        total = 0

        for child in children:
            total += edge_cost + ao_star(child)

        costs.append((total, connector_type, children))

    best = min(costs, key=lambda x: x[0])

    return best[0]

print("Total Minimum Cost from A:", ao_star('A'))
```

Output:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmatl
Total Minimum Cost from A: 2
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 4

Run All 5 Algorithms - See Where Each Works and Fails

Run DFS, BFS, Best First and A* on the same weighted graph (Program 2). Then run AO* on the AND-OR graph (Program 3). Compare paths, costs, and discover which properties (completeness, optimality) each satisfies in practice.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q4.py > bfs

```python
from collections import deque
import heapq
graph = {
    'S': [('A', 3), ('B', 5)],
    'A': [('C', 4), ('D', 2)],
    'B': [('D', 6), ('G', 9)],
    'C': [('G', 7)],
    'D': [('E', 1)],
    'E': [('G', 3)],
    'G': []
}
heuristic = {
    'S': 10,
    'A': 7,
    'B': 6,
    'C': 5,
    'D': 4,
    'E': 2,
    'G': 0
}
# DFS
def dfs(start, goal):
    stack = [(start, [start], 0)]
    visited = set()

    while stack:
        node, path, cost = stack.pop()

        if node == goal:
            return path, cost

        if node not in visited:
            visited.add(node)

            for neighbor, w in reversed(graph[node]):
                stack.append((neighbor, path + [neighbor], cost + w))
# BFS
def bfs(start, goal):
    q = deque([(start, [start], 0)])
    visited = set()

    while q:
        node, path, cost = q.popleft()

        if node == goal:
            return path, cost

        if node not in visited:
            visited.add(node)

            for neighbor, w in graph[node]:
                q.append((neighbor, path + [neighbor], cost + w))

# Best First Search
def best_first(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], start, [start], 0))

    visited = set()

    while pq:
        h, node, path, cost = heapq.heappop(pq)

        if node == goal:
            return path, cost

        if node not in visited:
            visited.add(node)

            for neighbor, w in graph[node]:
                heapq.heappush(
                    pq,
                    (heuristic[neighbor],
                     neighbor,
                     path + [neighbor],
                     cost + w)
                )

# A*
def astar(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0, start, [start]))

    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node == goal:
            return path, g

        if node not in visited:
            visited.add(node)

            for neighbor, w in graph[node]:
                new_g = g + w
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    pq,
                    (new_f,
                     new_g,
                     neighbor,
                     path + [neighbor])
                )

print("DFS:", dfs('S', 'G'))
print("BFS:", bfs('S', 'G'))
print("Best First:", best_first('S', 'G'))
print("A*:", astar('S', 'G'))
```

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desktop/Art
DFS: (['S', 'A', 'C', 'G'], 14)
BFS: (['S', 'B', 'G'], 14)
Best First: (['S', 'B', 'G'], 14)
A*: (['S', 'A', 'D', 'E', 'G'], 9)
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```